

Gizmo

Aidan Ryder

Mayli Jones



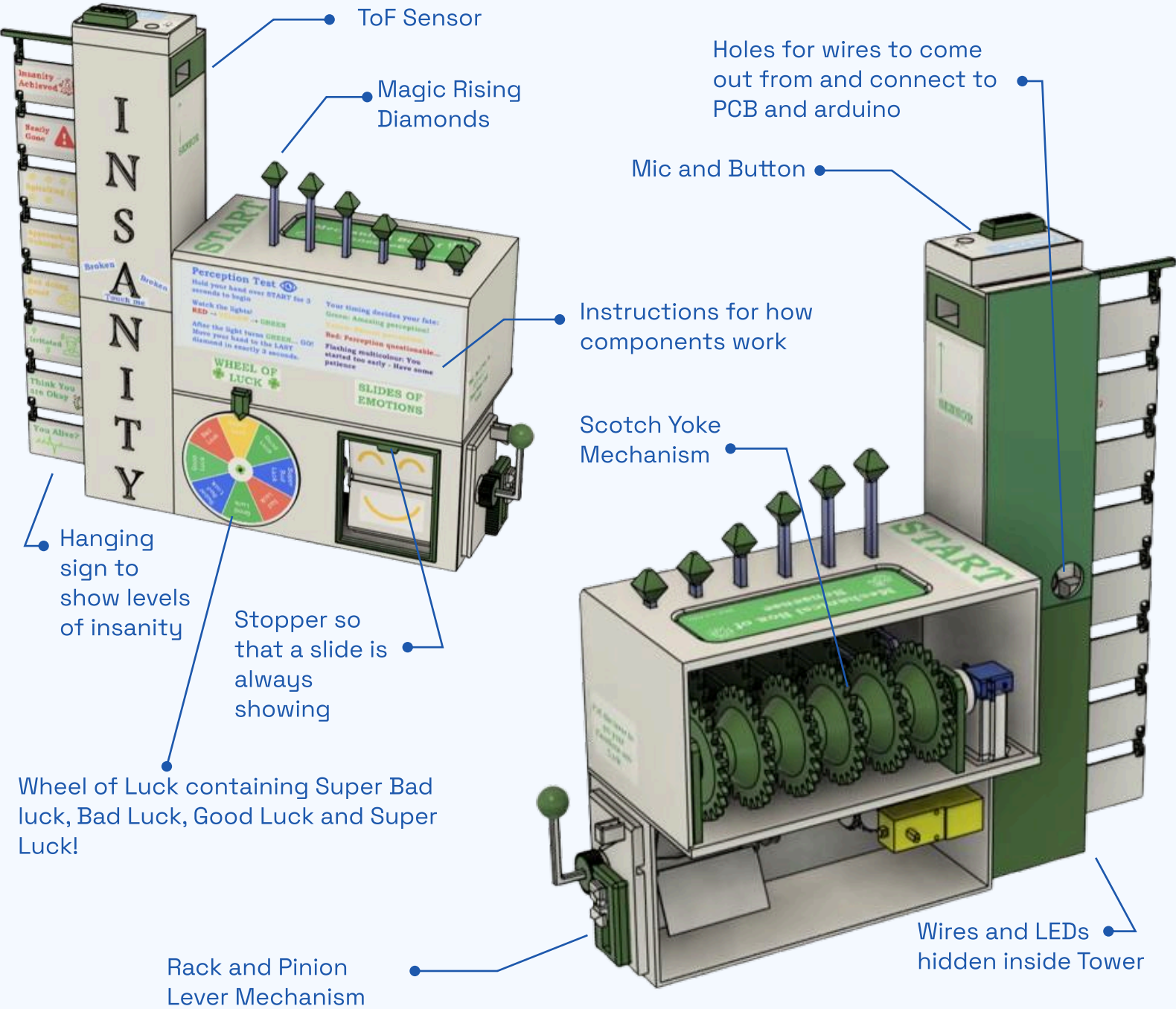
Mechanical Box of Nonsense

Project Motivation & Goals

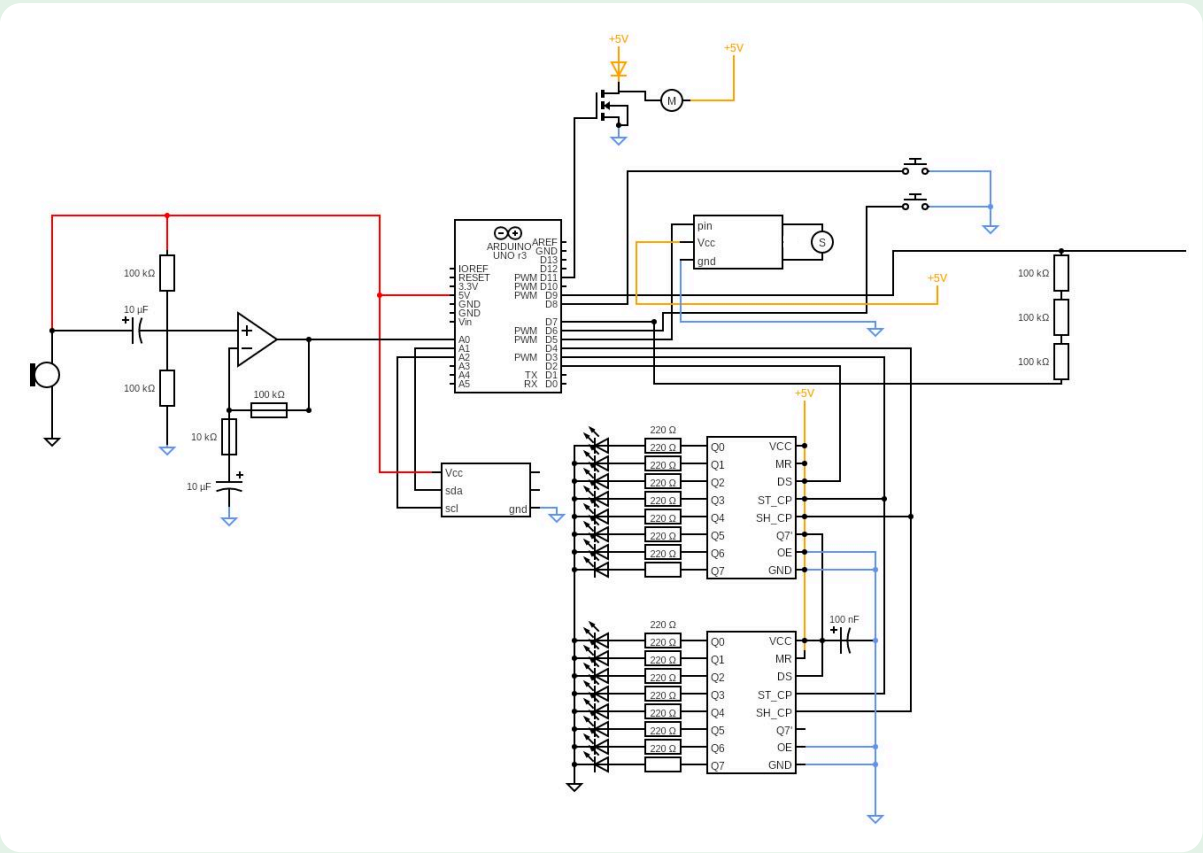
The concept is inspired by **RPGs** like Dungeons & Dragons, where characters are **defined by their stats**. Here, the Gizmo turns that idea into a **playful physical experience**: the box “measures” four stats, **Sanity, Perception, Luck, and Emotion**, using different input and output mechanisms.

Each stat is linked to a distinct reference or interaction style:

- **Luck & Emotion** borrow the feel of a **slot machine**, with the emotion mechanism based on a **flipper-clock** style rotating slide system.
- **Perception** is driven by a **crankshaft-inspired piston** mechanism, adapted from automotive engine layouts.
- **Sanity** takes cues from **carnival strength-test games**, translating loudness into an animated LED “insanity” bar



Full circuit Diagram



The full circuit diagram illustrates the **integration of sensors, actuators, and outputs** around a single microcontroller (Arduino UNO R3). **High-current** and **low-level signal** components are **separated** to **reduce electrical noise**, while **shift registers** are used to **minimise pin usage**. Protective components such as **decoupling capacitors** and **flyback diodes** are included to improve system reliability.

The **red and orange lines** both symbolise the **vcc** but red is from the Arduino power rail while orange is from the power board. This split is mostly **sensors** on the Arduino and then **motors** on the power board.

How do parts connect together?

The code is structured to be **modular**, using enums, functions, and switch-case **logic** to implement **multiple finite state machines**. **Separate state machines** control the pistons, insanity tower (lights/audio), and motor subsystems, making the behaviour **easier to debug, extend, and modify in future**.

Throughout the project we also tried to stay consistent with a theme of **resourcefulness**. Electrically, we **reduced the number of components** by using **mechanical design** to create motion and interaction instead of adding extra motors or sensors, and by fully **using the parts provided**. The **only additional components** purchased were **bearings and a ToF sensor**. We used **4** colours of LEDs to generate **7** different colours and patterns rather than buying an LED strip, and we based all LED electronics around the supplied PCB. Mechanically, all printed parts were designed to **minimise support material**, reducing filament waste and print time. Even the graphic elements were handled resourcefully, with stickers **designed, printed, and cut by hand** rather than outsourced.

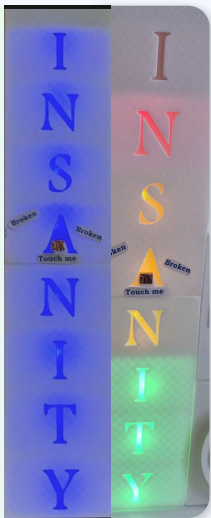
Insanity Tower

Interaction

The Insanity Tower is an **LED and audio based component** of our project, the **user screams** into a microphone which **triggers an LED bar to rise**.

In **idle**, the tower **glows blue** and the letter “A” **blinks** at random intervals. A capacitive touch sensor behind the “A” lets the user stop the blinking with a touch, giving a small interaction even when nothing else is running.

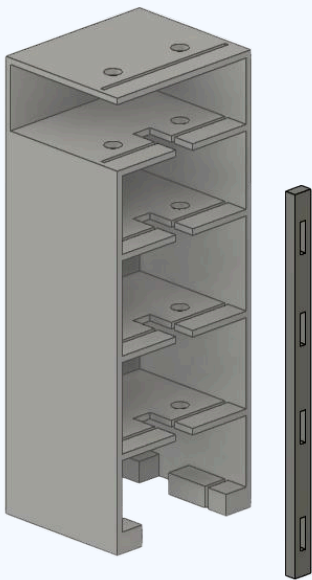
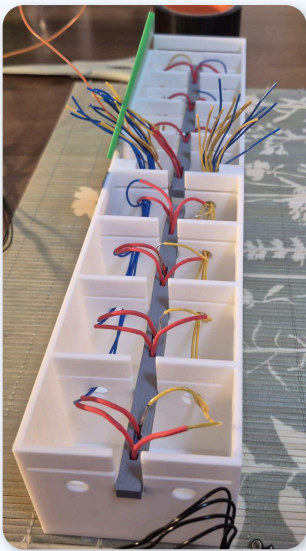
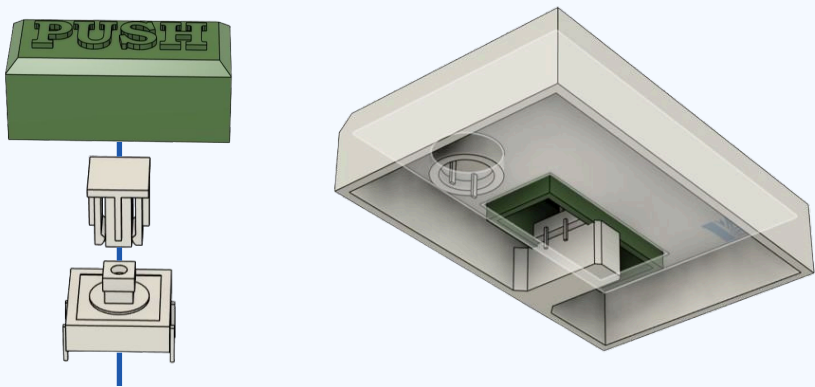
When the push button is pressed, the system runs a **“sanity check.”** The microphone measures peak volume and a bar of coloured **LEDs rises** to show the “insanity level.” If the sound passes a set threshold, the tower triggers an **extreme response** and goes **“Haywire”** in the rest of the mechanism.



Mechanisms

Button:

The top of the tower contains a push button that activates the microphone. Its holder was designed with **internal channels** so the **wires** can be **routed neatly** through the tower without being visible. The button cap itself is made from two 3D-printed parts: a small clip that attaches to the push button, and a larger “PUSH” cap that is glued onto the clip. This avoids gluing directly to the electronic button, making it easier to replace or repair if anything fails.

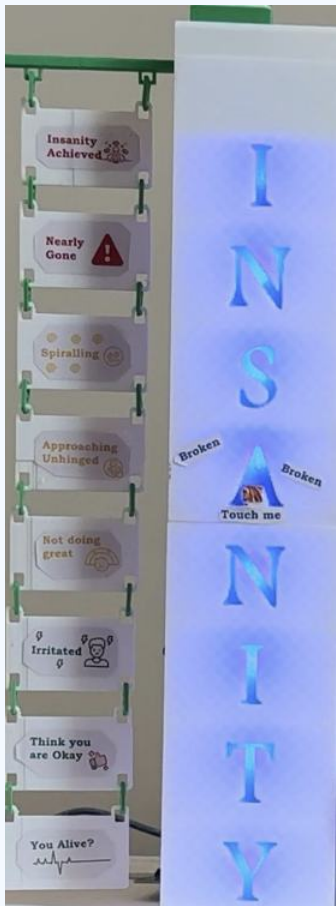


Wire/LED Considerations:

The tower uses 16 LEDs, creating 32 LED wires (38 wires total), so **internal tunnels** and LED frames were added to keep routing neat. LEDs were soldered to short leads and then connected via jumper wires to the PCB, so sections can be unplugged and **removed easily for repair**.

Hanging Signs:

To fully showcase levels of insanity and to add **humour**, chained signs were designed to hang from the side of the tower, each sign has a different **description for the users sanity** and also starts with the letter corresponding to the word Insanity. eg) I = irritated



Circuit & Code

LED Circuit:

The LED system uses **2 shift registers** to control **16 LEDs** using only **3 microcontroller pins**. The registers are driven with binary data represented as hexadecimal values in the code.

```
127 digitalWrite(latchPin, LOW);
128 shiftOut(dataPin, clockPin, MSBFIRST, 0xff);
129 shiftOut(dataPin, clockPin, MSBFIRST, 0x00);
130 digitalWrite(latchPin, HIGH);
```

OP-Amp circuit:

The microphone circuit has a **gain of 11x**. The microphone output is read by the Arduino as a value between 0–1023, corresponding to 0–5 V. While active, the code **behaves like a software peak detector**: it records the highest reading as the peak, converts this into a level, and uses a smoothing function to drive the LEDs. The peak is then gradually decayed so the lights fall smoothly rather than switching off instantly.

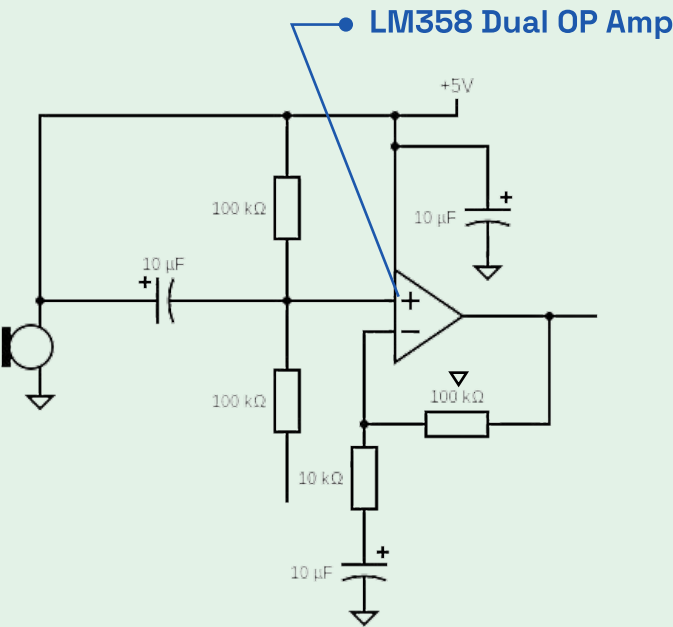
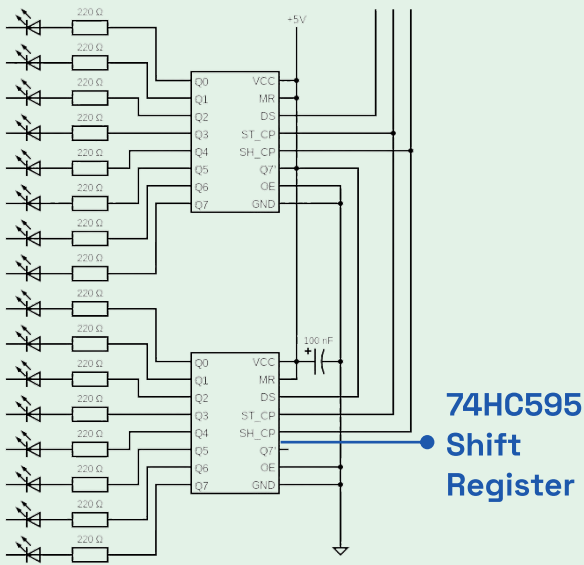
```
146 int LevelFromPeak(int p) {
147   if (p < 500) return 0;
148   if (p < 540) return 1;
149   else if (p < 560) return 2;
150   else if (p < 580) return 3;
151   else if (p < 600) return 4;
152   else if (p < 620) return 5;
153   else if (p < 650) return 6;
154   else if (p < 670) return 7;
155   else return 8;
156 }
```

Capacitors:

AC coupling: This is used to remove any DC offset from the microphone signal before applying a standardised 2.5 V bias to the system.

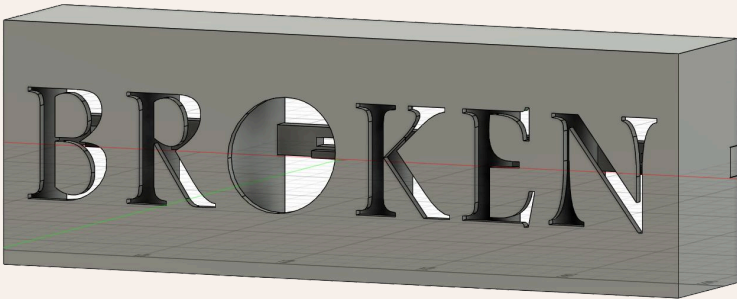
Grounding / decoupling: Noise from the Arduino power rail was reduced by using a capacitor to decouple the supply and stabilise the signal.

Gain control: A capacitor is used in the amplifier stage to block DC gain, ensuring only the AC audio signal is amplified.



Design Progression

In the original concept, we had a broken sign with the capacitive touch sensor, and a separate loudness/Sanity check. These two **concepts were combined** to create a one tower that does both at the same time. This was done to **reduce the amount of large prints** that would be done as well as making our entire design more **cohesive and integrated as one idea**.



Magic Diamonds

Interaction

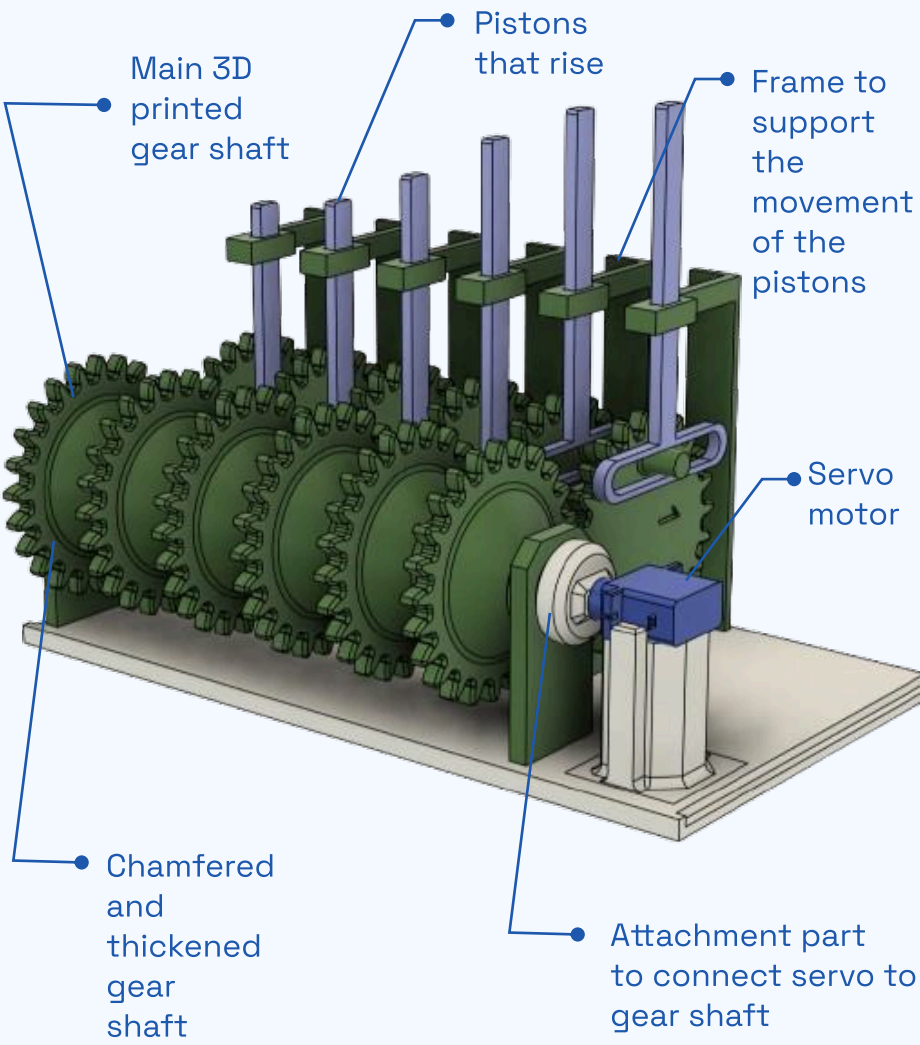
The Magic Diamonds pistons create a **reactive, skill-based game** where the pistons follow the user's hand.

In **idle mode**, the pistons move to where the users hand is by **mapping distances** from the ToF sensor and the hand itself, making the motion **feel intuitive and responsive**.

In **perception-test mode**, the user holds their hand above the start sticker, after 1-second a **countdown is shown** on the Insanity Tower (**red, yellow, green**). When the lights go off, the user must **move their hand** smoothly to the end of the pistons in **exactly 3** seconds. The system then grades their timing with a red, yellow or green result **shown with LEDs on the tower**. An early or **impatient movements** trigger a **“false start”** flash pattern that turns the LEDs different colours. This turns a simple sensing setup into a **small, timing-based challenge** with clear motion and colour feedback.



Mechanism - Scotch Yoke



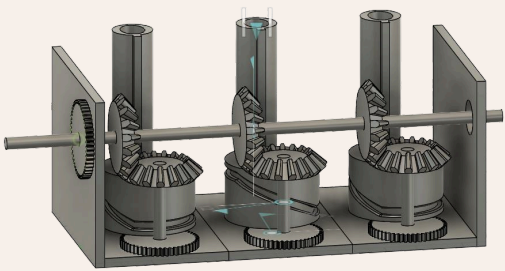
A **single servo motor drives** a 3D-printed main gear shaft. This shaft meshes **1:1 with six identical gears**, each acting as a Scotch-yoke driver for one piston: a pin on each gear runs in a slot in the piston carrier, so one full revolution is converted into smooth vertical motion of the diamond-topped piston. Because all six gears share the same shaft, the **pistons are synchronised** and always driven from the same input, which keeps the motion consistent and easy to control in code.

The lifting pins only **drive each piston over half a revolution**, and the gears are pre-phased around the shaft so no two pistons reach maximum height together. This creates a travelling **“wave” of motion** along the row of pistons, making the interaction feel more dynamic and alive. Using a single servo instead of six separate motors also reduces wiring, control complexity and material use, making the system more reliable and resource-efficient

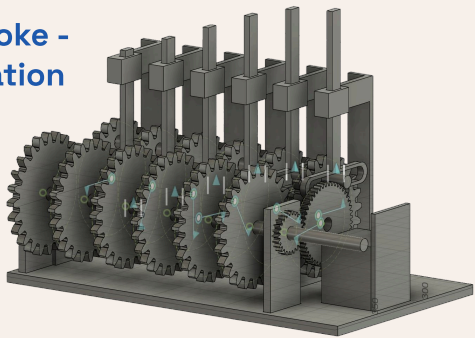
Design Iterations

The original concept was a **barrel cam system**, this system was tested through multiple iterations, we discovered that there was **too much friction** for the cam followers to move up the barrel especially with them being 3d printed so we **pivoted to a scotch yoke** mechanism. Our first designs of the scotch yoke mechanism showed the shaft was too weak and needed significant improvement.

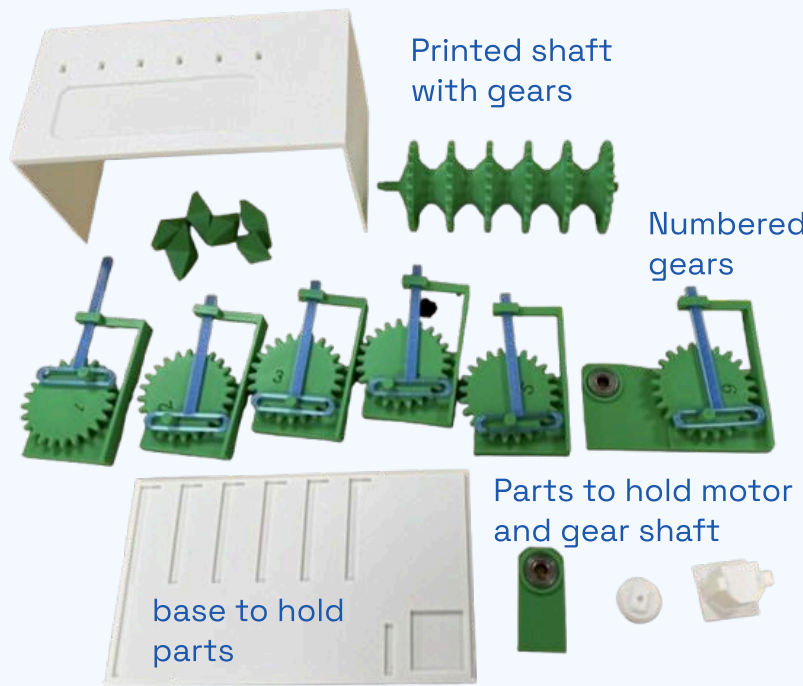
Barrel cam system



Scotch Yoke - first Iteration



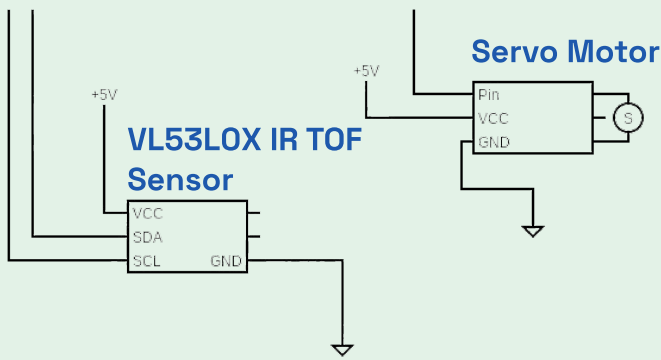
Assembly of Mechanism



Assembly was a **key design consideration**. The gear shaft is printed as a single part so the gears are **automatically aligned at their correct phase angles**, with chamfered connections for strength. The 40 mm piston stroke is set by placing the crank pins 20 mm from the gear centres (stroke = 2r), and each gear is engraved with a number so builders can quickly identify positions and avoid assembly errors.

Electronics

```
38 void AngleFromDistance(int d) {
39
40   if (d < 0 || d > 250) Angle = 0;
41   else {
42     d = constrain(d, 0, 180);
43     Angle = map(d, 0, 180, 0, 180);
44   }
45   Angle = constrain(Angle, 0, 180);
46   S.write(Angle);
47 }
```



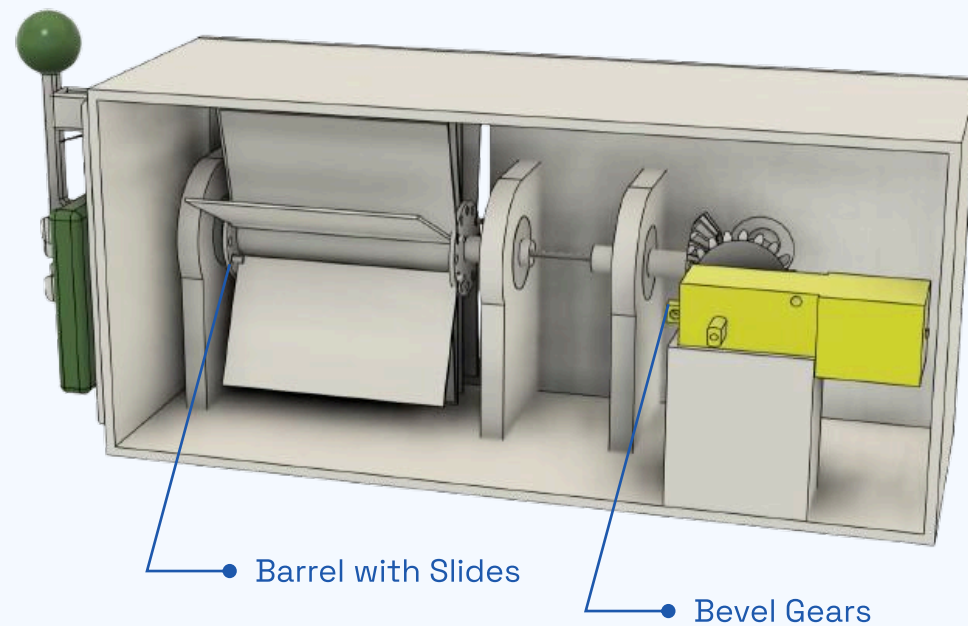
The piston system uses **continuous distance sensing** with from a ToF sensor mapping functions to drive a servo in real time. Motion is constrained in software to protect the hardware. **Millis was used to add non blocking timing** and state-dependent (**Enums, cases and state switches**) logic enable the piston to respond differently during idle, testing, and failure states.

Wheel of Luck & Slides of Emotion

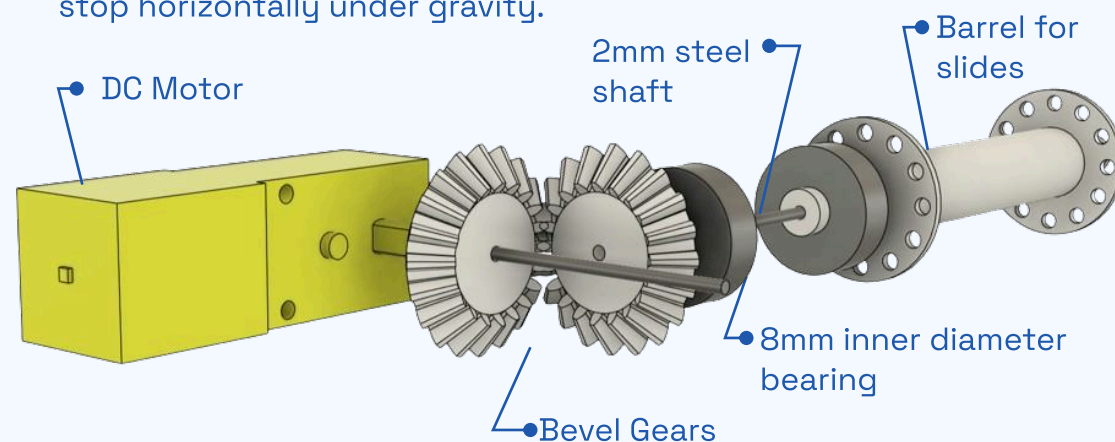
Interaction

The **Wheel of Luck** and **Slides of Emotion** randomly choose both your “luck for the day” and your “emotion for the day.” The user **pulls the side lever** to start the experience: while the lever is **held down**, the wheel and emotion sliders **spin continuously** and the Insanity Tower **LEDs change colours**. When the lever is **released**, the motion continues for a **random** amount of time before **slowing to a stop**, revealing the final luck and emotion combination.

Mechanisms - Wheel and Slides



- A DC motor **drives a thin metal shaft**; this shaft is directly attached to the wheel, spinning it.
- Halfway along the shaft, a **pair of bevel gears** with the same number of teeth create a **1:1 drive** to a perpendicular shaft. This **second shaft spins a barrel**, and the emotion slides are loosely attached around this barrel so they can **swing and move** as it rotates.
- As the barrel spins, the slides are carried round in a loop. A **fixed stopper** at the front **catches the top slide** when the motor stops, so **one slide** is always held **clearly in view** and doesn't fall back down or stop horizontally under gravity.



Mechanisms - Lever

The lever drives a small 3D-printed **pinion gear** which meshes with a **vertical rack**. At the top of the rack's travel there is a **limit switch**; when the rack hits this switch it tells the electronics that the **lever is fully pulled**, so the system can start the spin and LED sequence.

The rack is printed at **100% infill** so it is **heavier** than the lever, while the lever is printed at **10% infill** to keep it **light**. This weight difference helps the **lever return to its resting position** after being pulled. The rack is guided by two **vertical support posts** with caps on top, which stop it from slipping out of its guides or lifting off during use.

To keep the wiring tidy, a **shallow channel** was designed into the **back panel** behind the limit switch so the **wires can be routed and hidden** out of sight, improving both safety and aesthetics.

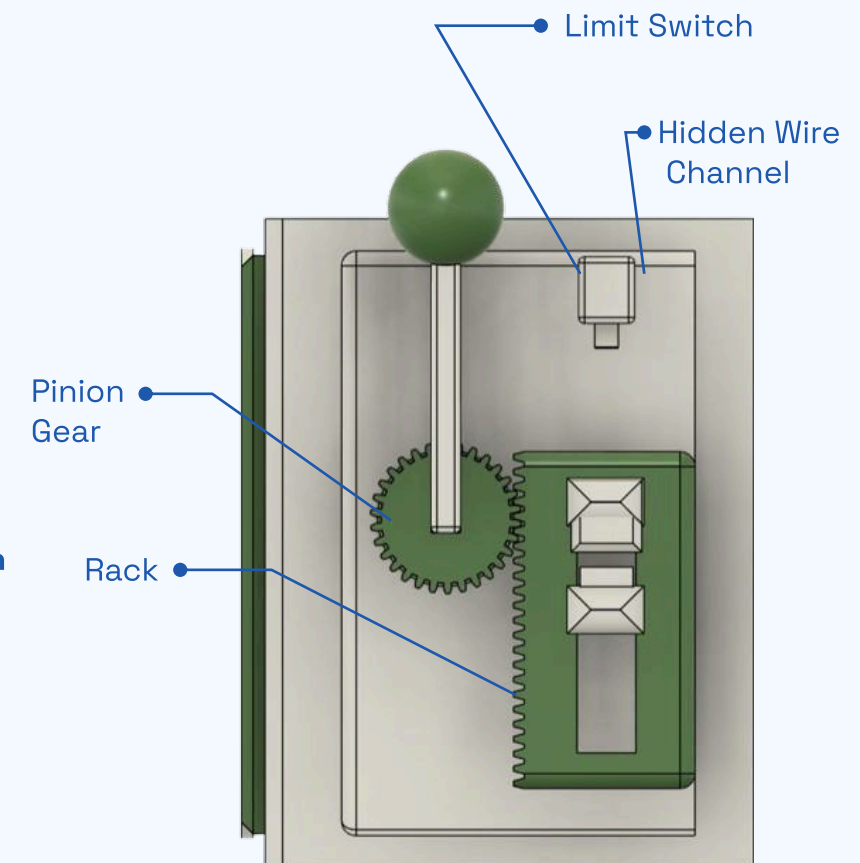
The rack was designed to **rise** by **approximately 20 mm** for a full lever pull, corresponding to a **90° rotation** of the lever. The required pinion size was determined using the **arc-length relationship**:

$$s=r\theta$$

For a rack to travel $s=20\text{mm}$ and Lever rotation $\theta = 90^\circ = \pi/2 \text{ rad}$:

$$r = s/\theta = 20/(\pi/2) = 12.8\text{mm}$$

A standard **module 0.8** gear with **32 teeth** was selected, giving a pitch diameter of **25.59 mm**. This closely matches the calculated requirement and provides the 20 mm rack travel needed for the rack to reach the limit switch at full lever pull



Design Iterations

The emotion slides were initially driven through a **2:1 gear ratio** so they would spin faster than the wheel. **Testing** showed that the slides already **rotated quickly enough** with a direct drive, and the extra gear added **unnecessary complexity** and material use. The intermediate gear was therefore **removed**, and the **barrel** was **redesigned** to fit directly onto the motor shaft, **simplifying the mechanism** and making it more resource-efficient.

Electronics

Limit switch: The switch logic was inverted, with a HIGH value in code behaving as LOW in hardware and vice versa, so this was handled in software.

MOSFET: A MOSFET was used instead of an H-bridge since only one direction of motor movement was needed.

Diode: A diode was added to prevent backflow and protect the circuit from voltage spikes.

Power railing: The motor and sensors were placed on separate power rails due to noise issues; when combined, the mic became unstable and the motor would not switch off correctly.

TT motor: This motor was chosen because the barrel required high torque due to its weight.

